

MeetingVoice

Guía completa para empaquetar tu PWA como APK Android con Trusted Web Activity (TWA)

100% gratuito · Sin barras de navegador · App nativa

Esta guía te explica, paso a paso, cómo convertir tu PWA ya funcional en un APK instalable en cualquier móvil Android. No necesitas aprender Kotlin ni reescribir tu app: la envolvemos con una capa nativa que abre tu web como si fuera una aplicación más del sistema.

Tiempo estimado	Dificultad	Coste	Resultado
30-60 min	Baja	0 €	APK + AAB firmados

Índice

- 1. ¿Qué es una TWA y por qué te conviene? — Concepto y ventajas
- 2. Requisitos previos — Lo que necesitas tener instalado
- 3. Estructura del proyecto — Qué contiene cada archivo
- 4. Paso 1: Instalar Android Studio — En Windows, Mac y Linux
- 5. Paso 2: Abrir el proyecto MeetingVoice-TWA — Sincronización Gradle
- 6. Paso 3: Configurar tu URL — Apuntar a tu PWA publicada
- 7. Paso 4: Probar en tu móvil o emulador — Ejecutar la app
- 8. Paso 5: Generar APK firmado — Para distribuir
- 9. Paso 6: Verificación de dominio — assetlinks.json (CRÍTICO)
- 10. Paso 7: Instalar el APK en tu móvil — Sin Play Store
- 11. Publicar en Google Play Store — Si quieres llegar a todos
- 12. Solución de problemas — Errores comunes
- 13. Personalización — Cambiar colores, icono, nombre

1. ¿Qué es una TWA y por qué te conviene?

Una Trusted Web Activity (TWA) es una tecnología oficial de Google que permite empaquetar una Progressive Web App (PWA) como si fuera una aplicación nativa de Android. La app resultante se instala con su propio icono en el escritorio del móvil, se abre a pantalla completa sin la barra de direcciones del navegador, aparece en el menú de apps recientes y puede solicitar permisos nativos como el micrófono. Para el usuario final es indistinguible de una app nativa escrita en Kotlin o Java.

La clave de una TWA es que no reescribe tu web: internamente usa Chrome (en modo incógnito de UI) para renderizar tu PWA, pero lo hace de forma transparente. Esto significa que cualquier cambio que hagas en tu web se refleja automáticamente en la app, sin tener que volver a compilar o publicar un APK nuevo. Solo necesitas actualizar la PWA en tu servidor y la app mostrará la versión nueva la próxima vez que se abra.

Ventajas frente a otras opciones

- Cero reescritura de código: aprovechas todo el trabajo que ya has hecho en Next.js, React, Tailwind.
- Una sola codebase: mantienes tu web y la app Android con los mismos cambios.
- Acceso a APIs nativas: micrófono, cámara, geolocalización, notificaciones push (vía Firebase).
- Instalación estándar: el APK se instala como cualquier app, sin pasar por Chrome.
- Google Play Store: puedes publicarlo sin problema, es una técnica oficialmente soportada.
- Coste cero: las herramientas son gratuitas y open source.

Limitaciones a tener en cuenta

- Necesita Chrome instalado en el móvil del usuario (está en el 95% de los Android).
- Requiere conexión a internet para cargar la app la primera vez (luego funciona offline si tu PWA tiene service worker).
- Para que no aparezca la barra de URL, debes subir un archivo `assetlinks.json` a tu servidor.
- Algunas APIs nativas avanzadas (NFC, Bluetooth BLE) requieren una capa Kotlin adicional.

2. Requisitos previos

Antes de empezar necesitas tener instalado Android Studio y publicado tu PWA en internet. Si todavía no has publicado tu PWA, en esta misma guía te explico cómo hacerlo gratis con Vercel.

2.1 Instalar Android Studio

Android Studio es el IDE oficial de Google para desarrollar apps Android. Es gratuito y funciona en Windows, Mac y Linux. La instalación incluye el Android SDK, un emulador de Android, y todas las herramientas necesarias para compilar APKs.

Descarga desde:

```
https://developer.android.com/studio
```

Durante la instalación, asegúrate de marcar Android SDK y Android Virtual Device (AVD). Si tu equipo tiene menos de 8 GB de RAM, el emulador puede ir lento; en ese caso, prueba directamente en tu móvil real conectado por USB.

2.2 Publicar tu PWA en internet (gratis, con Vercel)

Para que la TWA funcione, tu PWA tiene que estar accesible públicamente por HTTPS. La forma más rápida y gratuita es usar Vercel, que en 2 minutos te da una URL como <https://meetingvoice.vercel.app>.

```
# 1. Instala Vercel CLI en tu PC (solo la primera vez)
npm i -g vercel

# 2. Entra en la carpeta de tu proyecto MeetingVoice
cd /ruta/a/MeetingVoice

# 3. Publica (responde a las preguntas con Enter)
vercel

# 4. Te dará una URL como: https://meetingvoice-xxx.vercel.app
# 5. Para producción (URL definitiva):
vercel --prod
```

Alternativas gratuitas a Vercel: Netlify, Cloudflare Pages, GitHub Pages. Todas ofrecen HTTPS automático y deploy desde git.

3. Estructura del proyecto MeetingVoice-TWA

El proyecto que te he generado en `download/MeetingVoice-TWA/` contiene todo lo necesario para compilar un APK. Aquí te explico qué hace cada archivo:

```
MeetingVoice-TWA/
├── app/
│   ├── src/main/
│   │   ├── AndroidManifest.xml ← Identidad de la app
│   │   ├── java/com/meetingvoice/app/
│   │   │   └── MainActivity.kt ← Punto de entrada (Kotlin)
│   │   └── res/
│   │       ├── drawable/ ← Iconos vectoriales
│   │       ├── mipmap-anydpi-v26/ ← Icono adaptativo
│   │       ├── values/ ← strings, colors, themes
│   │       └── xml/filepaths.xml ← Config FileProvider
│   ├── build.gradle.kts ← Dependencias del módulo
│   └── proguard-rules.pro
├── .well-known/
│   └── assetlinks.json ← Subir a tu servidor HTTPS
├── build.gradle.kts ← Configuración raíz
├── settings.gradle.kts
├── gradle.properties
├── gradlew / gradlew.bat ← Scripts wrapper
└── README.md
```

3.1 AndroidManifest.xml — El carnet de identidad de la app

Es el archivo más importante. Define el package name único de tu app (`com.meetingvoice.app`), los permisos que necesita (micrófono, internet), la actividad principal que se abre al pulsar el icono y la URL de tu PWA. Es el único archivo que tendrás que editar para poner tu URL.

3.2 MainActivity.kt — Punto de entrada en Kotlin

Esta clase hereda de `LauncherActivity` (de la librería `androidbrowserhelper` de Google) que se encarga de abrir Chrome de forma transparente. Antes de abrir la TWA, nuestra clase pide permiso de micrófono al usuario — sin él, la PWA no podría grabar audio. Si el usuario lo rechaza, muestra un diálogo con un botón para abrir Ajustes del sistema y activarlo manualmente.

3.3 build.gradle.kts — Dependencias y configuración

Define la versión mínima de Android (7.0), la versión objetivo (Android 14), el package ID y las dependencias. La principal es `androidbrowserhelper`, de Google, que aporta toda la lógica TWA. No necesitas tocar este archivo salvo que quieras añadir librerías nativas adicionales.

3.4 assetlinks.json — Verificación de dominio

Archivo CRÍTICO. Sin él, Chrome mostrará una barra con tu dominio encima de la app, rompiendo la ilusión de app nativa. Debes subirlo a tu servidor en la ruta <https://tu-dominio.com/.well-known/assetlinks.json>. Contiene tu package ID y la huella SHA256 del certificado con el que firmas el APK. Más adelante te explico cómo generarlo.

4. Paso 1: Instalar Android Studio

Si todavía no tienes Android Studio, descárgalo de:

```
https://developer.android.com/studio
```

Sigue el instalador gráfico. En Windows acepta instalar los drivers USB de Google. En Mac descomprime y arrastra a Aplicaciones. En Linux descomprime en /opt y crea un acceso directo. La primera vez que abras Android Studio te pedirá descargar el SDK de Android (unos 2 GB) — acepta los valores por defecto.

Requisitos de hardware: 8 GB de RAM (16 GB recomendados), 6 GB de disco libre, procesador de 64 bits. Funciona en Intel y Apple Silicon (M1/M2/M3) nativamente.

5. Paso 2: Abrir el proyecto MeetingVoice-TWA

Copia la carpeta MeetingVoice-TWA a una ubicación fácil de encontrar (por ejemplo C:\Proyectos\ en Windows o ~/Proyectos/ en Mac/Linux). Después:

- Abre Android Studio.
- Pulsa Open (o File → Open).
- Selecciona la carpeta MeetingVoice-TWA (la raíz, no una subcarpeta).
- Pulsa OK y espera.

Android Studio empezará a sincronizar Gradle. La primera vez tarda entre 3 y 10 minutos porque descarga todas las dependencias. Verás una barra de progreso abajo. Cuando termine, aparecerá el mensaje 'Gradle sync finished' en la esquina inferior derecha. Si te pide instalar una versión específica del SDK, acepta.

Si la sincronización falla: comprueba tu conexión a internet. Si el problema persiste, pulsa File → Sync Project with Gradle Files. En último caso, cierra Android Studio, borra la carpeta .gradle dentro del proyecto y vuelve a abrir.

6. Paso 3: Configurar tu URL

Ahora hay que decirle a la app dónde está publicada tu PWA. Abre el archivo `app/src/main/AndroidManifest.xml` y busca estas dos líneas (están en distintos puntos del archivo):

```
<meta-data
  android:name="android.support.customtabs.trusted.DEFAULT_URL"
  android:value="https://tu-dominio.com" /> ← CAMBIA ESTO

...

<data
  android:scheme="https"
  android:host="tu-dominio.com" /> ← Y ESTO TAMBIÉN
```

Reemplaza `tu-dominio.com` por tu dominio real. Por ejemplo, si tu PWA está en `https://meetingvoice.vercel.app`, las líneas quedan así:

```
<meta-data
  android:name="android.support.customtabs.trusted.DEFAULT_URL"
  android:value="https://meetingvoice.vercel.app" />

<data
  android:scheme="https"
  android:host="meetingvoice.vercel.app" />
```

Guarda el archivo (Ctrl+S / Cmd+S). Android Studio puede pedirte volver a sincronizar Gradle — acepta.

Importante: la URL debe ser HTTPS. Las TWA no permiten HTTP salvo en modo debug y para dominios muy específicos. Vercel, Netlify y todos los hosting modernos dan HTTPS gratis.

7. Paso 4: Probar en tu móvil o emulador

Antes de generar el APK final, vamos a probar que todo funciona. Tienes dos opciones:

Opción A — En tu móvil Android real (recomendada)

- En tu móvil: Ajustes → Acerca del teléfono → pulsa 7 veces sobre Número de compilación.
- Aparecerá el mensaje 'Ya eres desarrollador'.
- Vuelve atrás: Ajustes → Sistema → Opciones de desarrollador → activa Depuración USB.
- Conecta el móvil al PC con un cable USB (mejor el original, no uno solo de carga).
- En el móvil aparecerá '¿Permitir depuración USB?' → pulsa Aceptar.
- En Android Studio, arriba verás un desplegable con los dispositivos conectados. Selecciona tu móvil.
- Pulsa el botón verde ► Run (o Mayús+F10).
- La app se compila (1-3 min la primera vez), se instala en tu móvil y se abre automáticamente.

Opción B — En el emulador de Android Studio

- En Android Studio: Tools → Device Manager.
- Pulsa Create Device.
- Elige un teléfono, por ejemplo Pixel 7 → Next.
- Descarga una imagen de Android (recomendado: API 34, Android 14) → Next → Finish.
- Pulsa el botón verde ► Run. La app se compila y se abre en el emulador.

Nota sobre el emulador: el micrófono virtual puede no funcionar bien. Para probar la grabación real de audio usa siempre un móvil físico. El emulador sirve para validar que la app abre, se ve bien y los permisos se piden correctamente.

Cuando la app se abra por primera vez, te pedirá permiso de micrófono. Pulsa Permitir. Verás tu PWA cargarse a pantalla completa, sin barra de direcciones. ¡Enhorabuena! Ya tienes MeetingVoice funcionando como app nativa.

8. Paso 5: Generar APK firmado para distribuir

El APK que se genera al pulsar Run está firmado con una clave de debug que no vale para distribuir. Para tener un APK que cualquier usuario pueda instalar necesitas firmarlo con un keystore propio. Sigue estos pasos:

- En Android Studio: menú Build → Generate Signed App Bundle / APK.
- Selecciona APK (no App Bundle) → Next.
- En Key store path pulsa Create new.
- Elige dónde guardar el archivo .jks (por ejemplo C:\Keystores\meetingvoice.jks).
- Rellena todos los campos: contraseña del keystore, contraseña de la clave, alias (por ejemplo meetingvoice), validez (25 años), tu nombre, organización, etc.
- GUARDA ESTOS DATOS: si pierdes la contraseña o el keystore, no podrás actualizar la app en Google Play. Haz una copia de seguridad del archivo .jks en Drive/Dropbox.
- Pulsa OK → Next.
- Selecciona release → Finish.
- Android Studio compila (2-3 min) y muestra un popup azul: 'locate'. Pulsa ahí para abrir la carpeta con tu APK.

El APK estará en `app/build/outputs/apk/release/app-release.apk`. Este es el archivo que puedes pasar a cualquier persona para que lo instale en su móvil.

Importante sobre el keystore: genera UNO y úsalo SIEMPRE para todas las versiones de tu app. Nunca generes uno nuevo, porque Google Play no te dejará actualizar la app con un keystore distinto. Haz copia de seguridad del archivo .jks y de la contraseña en sitios seguros (1Password, KeePass, papel en una caja fuerte).

9. Paso 6: Verificación de dominio (assetlinks.json)

Si pruebas tu APK ahora, verás que Chrome muestra una barra superior con tu dominio. Para eliminarla y conseguir una experiencia 100% nativa, necesitas subir un archivo `assetlinks.json` a tu servidor HTTPS.

9.1 Generar tu huella SHA256

Abre una terminal y ejecuta (sustituye la ruta por la de tu keystore):

```
keytool -list -v -keystore C:\Keystores\meetingvoice.jks -alias meetingvoice
```

Te pedirá la contraseña del keystore.
Busca la línea que dice:
SHA256: XX:X
X:XX:XX:XX:XX:XX:XX
Cópiala completa (con los dos puntos).

9.2 Crear el archivo assetlinks.json

Abre el archivo `.well-known/assetlinks.json` del proyecto y reemplaza `REEMPLAZAR_CON_TU_HUELLA_SHA256` por la huella que acabas de obtener. El archivo debe quedar así:

```
[
{
"relation": ["delegate_permission/common.handle_all_urls"],
"target": {
"namespace": "android_app",
"package_name": "com.meetingvoice.app",
"sha256_cert_fingerprints": [
"AB:CD:EF:12:34:56:78:90:AB:CD:EF:12:34:56:78:90:AB:CD:EF:12:34:56:78:90:AB:CD:EF:12:34:56:78:90"
]
}
}
]
```

9.3 Subirlo a tu servidor

El archivo debe ser accesible públicamente en esta URL EXACTA:

```
https://tu-dominio.com/.well-known/assetlinks.json
```

En Next.js, colócalo en `public/.well-known/assetlinks.json` y se servirá automáticamente. En Vercel/Netlify funciona sin configuración adicional. Verifica abriendo la URL en el navegador — debe devolverte el JSON.

Verificación: tras subir el archivo, espera 5 minutos y reinstala tu APK. Si todo está correcto, la barra de dominio habrá desaparecido y la app se verá 100% nativa.

10. Paso 7: Instalar el APK en tu móvil

Una vez generado `app-release.apk`, puedes instalarlo en cualquier móvil Android sin necesidad de Play Store. Tienes varias formas de pasarlo al móvil:

- Email: envíate el APK por Gmail. Ábrelo desde el móvil y pulsa 'Instalar'.
- Google Drive / Dropbox: súbelo, compártelo con tu cuenta, descárgalo en el móvil.
- USB: copia el archivo al almacenamiento del móvil y ábrelo desde el explorador de archivos.
- ADB: con depuración USB activa: `adb install app-release.apk`

La primera vez que intentes instalar un APK de origen desconocido, Android mostrará un aviso de seguridad. Esto es normal — Google solo avisa porque la app no viene de Play Store. Para permitirlo:

- Pulsa Ajustes en el aviso.
- Activa Permitir desde esta fuente (o Instalar apps desconocidas según el fabricante).
- Vuelve atrás y pulsa Instalar.
- Cuando termine, pulsa Abrir. ¡Listo!

A partir de aquí, MeetingVoice funciona como cualquier otra app: aparece en el cajón de apps, se puede desinstalar desde Ajustes, recibe actualizaciones cuando vuelvas a instalar un APK más moderno con el mismo package ID y mayor versionCode.

11. Publicar en Google Play Store (opcional)

Si quieres que cualquier persona pueda descargar tu app desde la tienda oficial de Google, estos son los pasos. El coste es de 25 € únicos (pago una sola vez en la vida) para crear la cuenta de desarrollador.

- Crea una cuenta en <https://play.google.com/console> (25 €).
- En Android Studio: Build → Generate Signed App Bundle / APK → esta vez elige Android App Bundle.
- Sube el archivo .aab a Google Play Console.
- Rellena la ficha de la tienda: descripción, capturas de pantalla (mínimo 2), icono 512×512, banner 1024×500, categoría, política de privacidad.
- Configura la firma de Play App Signing (Google te pide subir tu keystore de release una sola vez, ellos firman las versiones futuras con su clave).
- Sube el assetlinks.json a tu servidor (igual que en el paso 6).
- Envía para revisión. Google tarda entre 1 y 7 días en aprobar.

Política de privacidad obligatoria: Google exige que tengas una URL pública con tu política de privacidad. Si tu app no recoge datos personales más allá del micrófono (que es tu caso), puedes usar generadores gratuitos como privacypolicies.com.

12. Solución de problemas

Aquí tienes los errores más comunes y cómo resolverlos:

Gradle sync failed

Comprueba conexión a internet. Si persiste: File → Sync Project with Gradle Files. En último caso, borra la carpeta `.gradle` del proyecto y vuelve a abrir.

SDK not found / license not accepted

Android Studio te lo dirá y ofrecerá instalarlo. Acepta. Para aceptar todas las licencias de golpe: `sdkmanager --licenses` en terminal.

La app abre pero se ve la barra de Chrome con mi dominio

Tu `assetlinks.json` no está accesible o la SHA256 no coincide. Verifica abriendo en el navegador `https://tu-dominio.com/.well-known/assetlinks.json` — debe devolver el JSON. La SHA debe ser exactamente la del keystore con el que firmaste el APK.

No se puede grabar audio / el botón no hace nada

El permiso de micrófono está denegado. Ve a Ajustes del móvil → Apps → MeetingVoice → Permisos → Micrófono → Permitir. Si el problema persiste, comprueba que estás usando Chrome (no Firefox u otro navegador).

La app se cierra al abrirla (crash)

Conecta el móvil por USB y ejecuta en terminal: `adb logcat | grep MeetingVoice` para ver el error exacto. Lo más común: URL mal configurada en el Manifest, o falta de Chrome instalado en el móvil.

No aparece mi móvil en el desplegable de dispositivos

Comprueba que la depuración USB está activada y el cable es de datos (no solo carga). Ejecuta `adb devices` en terminal — debería listar tu móvil.

Error INSTALL_FAILED_UPDATE_INCOMPATIBLE

Ya tienes una versión anterior instalada con un certificado distinto. Desinstala la vieja antes: `adb uninstall com.meetingvoice.app`.

El APK es muy grande (>10 MB)

Es normal para una TWA con iconos PNG. Si quieres reducirlo, activa `isMinifyEnabled = true` en `build.gradle.kts`.

13. Personalización

13.1 Cambiar el nombre de la app

Edita `app/src/main/res/values/strings.xml`:

```
<resources>
<string name="app_name">MeetingVoice</string>
<!-- Cámbialo por el nombre que quieras -->
</resources>
```

13.2 Cambiar los colores

Edita `app/src/main/res/values/colors.xml`. Los colores deben coincidir con los de tu PWA (`theme_color` y `background_color` del `manifest.json`):

```
<resources>
<color name="colorPrimary">#6366F1</color> ← color principal (violeta)
<color name="colorPrimaryDark">#4F46E5</color> ← barra de estado
<color name="colorAccent">#8B5CF6</color> ← acento
</resources>
```

13.3 Cambiar el icono de la app

Los iconos están en `app/src/main/res/drawable/` como XML vectorial (`ic_launcher_background.xml` e `ic_launcher_foreground.xml`). Puedes editarlos directamente o, si prefieres PNGs, sustitúyelos por imágenes en `mipmap-xxxhdpi/` en sus distintas resoluciones. La forma más rápida es usar Image Asset Studio: clic derecho en `res` → `New` → `Image Asset` → sube tu logo y Android Studio genera todas las resoluciones.

13.4 Cambiar el package ID

Si quieres cambiar `com.meetingvoice.app` por tu propio dominio (por ejemplo `com.tuempresa.meetingvoice`):

- Edita `app/build.gradle.kts`: `applicationId = "com.tuempresa.meetingvoice"`
- Edita `namespace = "com.tuempresa.meetingvoice"`
- Renombra la carpeta `java/com/meetingvoice/app/` a `java/com/tuempresa/meetingvoice/`
- Actualiza el package al inicio de `MainActivity.kt`
- Actualiza el `package_name` en `assetlinks.json`

Resumen final

Has convertido tu PWA en una app Android instalable. Recapitulando lo que has conseguido:

- Una app con icono propio en el cajón de aplicaciones.
- Sin barra de navegador, experiencia 100% nativa.
- Acceso al micrófono con diálogo de permisos nativo.
- APK distribuible por email, Drive, USB o Play Store.
- Todo gratis y con código abierto.
- Mantenimiento simplificado: actualizas la web y todos los usuarios ven la nueva versión.

¿Y ahora qué?

1. Personaliza icono, colores y nombre según tu marca.
2. Distribuye el APK a tus usuarios (email, Drive, web de descarga).
3. Si quieres más alcance, publícalo en Google Play Store.
4. Si en el futuro necesitas notificaciones push o acceso a contactos nativos, podemos añadir una capa Kotlin más amplia sin reescribir la PWA.

Para cualquier duda o si te atascas en algún paso, vuelve a esta guía y revisa la sección 12 de solución de problemas. Si el problema no está listado, copia el mensaje de error exacto y pídenos ayuda.